



Matrices dispersas

Proyecto final de curso

Análisis y diseño de algoritmos I

Ingeniería en Sistemas

Karen Sofía López Botero

202459519

Laura Sofía Echeverry González

202477067

Docente: Mateo Echeverry Correa

Universidad del Valle

Sede Tuluá, Valle del Cauca

Mayo de 2026

Índice

1. Descripción general del problema resuelto	2
2. Formato de entrada y salida	2
3. Diseño general de la solución	3
4. Estructuras de datos implementadas	3
5. Justificación de cada estructura	4
6. Operaciones soportadas	4
7. Análisis de complejidad temporal	5
8. Análisis de uso de memoria	6
9. Estrategia de dividir y vencer implementada	6
10. Comparación con una solución ingenua	7
11. Casos de prueba utilizados	7
12. Casos límite considerados	8
13. Conclusiones	9

1. Descripción general del problema resuelto

El proyecto implementa un procesador eficiente de matrices dispersas, correspondiente al problema 2 del enunciado del proyecto final de *Análisis y diseño de algoritmos I*. El objetivo es diseñar una solución algorítmica que permita trabajar con matrices de dimensiones potencialmente muy grandes, pero donde solo una fracción pequeña de las posiciones contiene valores distintos de cero o valores relevantes.

En lugar de construir una matriz densa de tamaño $F \times C$, lo cual sería inviable cuando las dimensiones están en el orden de 10^9 , la solución almacena únicamente las posiciones no nulas usando estructuras de datos propias. Sobre esta representación dispersa, el programa debe procesar operaciones de consulta y actualización, garantizando corrección en las salidas y un uso razonable del tiempo de ejecución y de la memoria.

El sistema lee los datos iniciales y las operaciones desde un archivo plano llamado `entrada.txt`, procesa las consultas en el orden en que aparecen y genera las respuestas en un archivo `salida.txt`, sin depender de entrada interactiva ni de interfaces gráficas. El diseño se centra en aplicar conceptos del curso como diseño de algoritmos eficientes, análisis de complejidad temporal, análisis de uso de memoria, implementación propia de estructuras de datos, procesamiento de archivos planos y uso de estrategias de dividir y vencer en una operación relevante.

2. Formato de entrada y salida

El formato de entrada sigue la estructura sugerida para el problema de matrices dispersas:

```
F C N
fila columna valor
fila columna valor
...
Q
OPERACION parametros
OPERACION parametros
...
```

En la primera línea, F representa la cantidad total de filas de la matriz, C la cantidad total de columnas y N la cantidad de posiciones no nulas iniciales. A continuación aparecen N líneas, cada una con una tripleta `fila columna valor` que indica la coordenada y el valor almacenado en esa posición.

Después de las posiciones iniciales, se encuentra una línea con el entero Q , que indica la cantidad de operaciones a procesar. Luego se listan Q líneas, cada una con una operación y sus parámetros, usando las palabras clave (GET, SET, DELETE, ROW_SUM, COL_SUM, REGION_SUM, TRANSPOSE, TOP_K y DENSITY), según la especificación del problema.

El archivo `salida.txt` contiene una línea por cada operación que genera resultado, usando un formato claro y verificable. Por ejemplo, para el caso de prueba ilustrativo del enunciado, una operación como `GET 500 300` produce una línea de salida del tipo `GET 500 300 = 9`, mientras que una operación `TOP_K 3` produce una línea como `TOP_K 3 = (1,100,20) (999999999,999999999,15) (200000,10,11)`.

3. Diseño general de la solución

La solución se organizó en módulos independientes dentro de la carpeta `src`, con responsabilidades claras y acopladas de forma mínima. El flujo principal inicia en `main.py`, donde se construyen las rutas a `entrada/entrada.txt` y `salida/salida.txt`, se verifica la existencia del archivo de entrada y se delega el procesamiento a una función central en el módulo `formateo.py`.

El módulo `formateo.py` es responsable de leer el archivo de entrada, interpretar el encabezado F , C , N , cargar los valores no nulos iniciales en una instancia de la clase `MatrizDispersa` y procesar las Q operaciones en el orden en que aparecen. Este módulo también genera las cadenas de salida con el formato especificado en el enunciado y las escribe en el archivo `salida.txt`.

La clase `MatrizDispersa`, definida en `matriz_dispersa.py`, encapsula la representación lógica de la matriz dispersa y proporciona métodos para cada operación: `get`, `set`, `delete`, `row_sum`, `col_sum`, `region_sum`, `transpose`, `top_k` y `density`. Internamente, `MatrizDispersa` delega el almacenamiento físico de los elementos no nulos a una estructura propia implementada en el módulo `lista_enlazada.py`.

El módulo `lista_enlazada.py` implementa una lista enlazada simple mediante las clases `Nodo` y `ListaEnlazada`, donde cada nodo almacena una tripleta (`fila`, `columna`, `valor`) y un puntero al siguiente nodo. Esta estructura permite representar únicamente las celdas relevantes de la matriz, evitando el uso de arreglos densos de tamaño $F \times C$.

Finalmente, el módulo `merge_sort.py` implementa un algoritmo de ordenamiento por *merge sort* en orden descendente, utilizado por `MatrizDispersa` para resolver la operación `TOP_K`. De esta manera, el diseño general integra lectura y escritura de archivos planos, una representación dispersa de la matriz y una estrategia explícita de dividir y vencer para una operación de ranking.

4. Estructuras de datos implementadas

En la solución se implementaron las siguientes estructuras de datos propias:

- **Nodo:** estructura básica que almacena una fila, una columna, un valor y una referencia al siguiente nodo, utilizada como unidad mínima de la lista enlazada.
- **ListaEnlazada:** lista enlazada simple que mantiene una referencia a la cabeza y un contador de tamaño, y ofrece operaciones para insertar o actualizar posiciones, buscar valores, eliminar nodos, convertir la estructura a una lista de tuplas y vaciar su contenido.

- **MatrizDispersa**: estructura de mayor nivel que modela una matriz dispersa sobre la `ListaEnlazada`, almacenando solamente las posiciones no nulas junto con las dimensiones globales F y C .

Estas estructuras permiten representar y manipular los datos dispersos sin recurrir a matrices completas ni a estructuras nativas prohibidas como `dict`, `set` o `sorted`, cumpliendo con las restricciones del proyecto. Además, la implementación de `merge sort` en un módulo separado actúa como estructura algorítmica de soporte para ordenar colecciones de elementos relevantes.

5. Justificación de cada estructura

La estructura **Nodo** se justifica como un componente mínimo necesario para construir listas enlazadas, donde cada posición no nula de la matriz se representa explícitamente con sus coordenadas y valor. Al almacenar (`fila`, `columna`, `valor`) dentro de cada nodo, se preserva toda la información necesaria para operaciones posteriores sobre filas, columnas y regiones.

La **ListaEnlazada** se eligió como estructura base porque permite almacenar únicamente las celdas no nulas sin requerir memoria proporcional al dominio completo de la matriz, cuyas dimensiones pueden ser del orden de 10^9 según el enunciado. Aunque el acceso a una posición específica tiene costo lineal en el número de elementos almacenados, esta representación cumple con la restricción de evitar matrices densas y es coherente con el enfoque de trabajar solo sobre posiciones relevantes.

La estructura **MatrizDispersa** sirve como capa de abstracción que oculta los detalles de la lista enlazada y expone métodos con nombres directamente relacionados con las operaciones del problema. Esto facilita separar el diseño algorítmico de la representación física, permite futuras mejoras (por ejemplo, índices adicionales) sin cambiar la interfaz externa y apoya la claridad del documento técnico y de la sustentación.

Por último, el uso de **merge sort** como estructura algorítmica para ordenar elementos en `TOP_K` se justifica porque el proyecto requiere explícitamente una estrategia de dividir y vencer integrada en una operación relevante, y porque `merge sort` ofrece complejidad $O(N \log N)$ estable y bien conocida. Esta elección permite comparar la solución implementada con un enfoque ingenuo de ordenamiento cuadrático y resaltar las ventajas del diseño propuesto.

6. Operaciones soportadas

La clase `MatrizDispersa` implementa las operaciones mínimas sugeridas para el problema de matrices dispersas:

- **GET fila columna**: busca en la lista enlazada un nodo con la fila y columna solicitadas. Si lo encuentra, devuelve su valor; si no existe, devuelve 0, coherente con el comportamiento de una matriz dispersa donde las posiciones no almacenadas se consideran cero.

- **SET fila columna valor**: si el valor es distinto de cero, inserta una nueva posición o actualiza la existente llamando a `insertar_o_actualizar` en la lista enlazada; si el valor es cero, elimina la posición mediante `eliminar`, evitando almacenar ceros explícitos.
- **DELETE fila columna**: elimina la posición indicada de la lista enlazada si existe, y deja la celda implícitamente en cero. La operación siempre reporta OK en la salida.
- **ROW_SUM fila**: recorre la lista enlazada y acumula los valores de todos los nodos cuya fila coincide con la fila solicitada. Esta operación implementa la suma de una fila sin necesidad de recorrer una matriz densa completa.
- **COL_SUM columna**: de forma análoga, recorre la lista enlazada y suma los valores de los nodos cuya columna coincide con la columna indicada.
- **REGION_SUM f1 c1 f2 c2**: recorre todos los nodos y, para cada uno, verifica si su fila está entre $f1$ y $f2$ y su columna entre $c1$ y $c2$; si el nodo está dentro del rectángulo, su valor se suma al total.
- **TRANSPOSE**: intercambia las coordenadas de cada nodo (fila pasa a columna y columna pasa a fila) y también intercambia las dimensiones globales F y C , implementando la transposición de la matriz sobre la representación dispersa.
- **TOP_K k**: convierte la lista enlazada en una lista de tuplas ($fila, columna, valor$), la ordena en orden descendente por valor usando `merge sort` y devuelve las primeras k posiciones más relevantes.
- **DENSITY**: calcula la densidad de la matriz como la razón entre el número de celdas no nulas almacenadas y el número total de celdas $F \times C$, devolviendo un valor en punto flotante.

El módulo `formateo.py` se encarga de traducir estas operaciones internas en líneas de texto con el formato requerido en el archivo de salida, asegurando que cada consulta produzca una respuesta clara, consistente y verificable.

7. Análisis de complejidad temporal

En esta sección se analiza la complejidad temporal de las operaciones principales, utilizando N para denotar el número de posiciones no nulas almacenadas en la matriz dispersa. Dado que la representación se basa en una lista enlazada simple sin índices auxiliares, muchas operaciones dependen de recorridos lineales sobre los nodos.

La operación **GET** recorre a lo más todos los nodos de la lista enlazada en el peor caso, por lo que su complejidad temporal es $O(N)$. De manera similar, las operaciones **SET** (cuando la posición ya existe), **DELETE**, **ROW_SUM**, **COL_SUM** y **REGION_SUM** requieren recorrer la lista para localizar posiciones o acumular valores, y también presentan complejidad $O(N)$ en el peor caso.

La operación **TOP_K** transforma primero la lista enlazada en una lista de tuplas, lo cual tiene costo $O(N)$, y luego aplica `merge sort` sobre esa lista, con complejidad $O(N \log N)$. Por tanto, el

costo total de TOP_K está dominado por el ordenamiento y es $O(N \log N)$, lo cual mejora claramente frente a un ordenamiento cuadrático ingenuo $O(N^2)$.

La operación **TRANSPOSE** recorre una única vez todos los nodos para intercambiar las coordenadas, lo que implica complejidad $O(N)$, y las operaciones sobre las dimensiones globales son $O(1)$. Finalmente, **DENSITY** se implementa como una operación $O(1)$, ya que solo lee el tamaño de la lista enlazada y calcula un cociente simple utilizando F y C .

8. Análisis de uso de memoria

El uso de memoria de la solución está dominado por el almacenamiento de los nodos de la lista enlazada y por las listas temporales utilizadas en TOP_K. Cada nodo almacena una fila, una columna, un valor y un puntero, por lo que la memoria utilizada crece en proporción directa al número de posiciones no nulas N , es decir, $O(N)$.

A diferencia de una matriz densa de tamaño $F \times C$, que requeriría memoria $O(F \cdot C)$ y sería inviable cuando F y C son del orden de 10^9 , la representación dispersa evita reservar espacio para posiciones que nunca se utilizan. Esto cumple con la restricción explícita del proyecto de no construir matrices completas ni estructuras proporcionales al dominio total cuando este puede ser muy grande.

Durante la operación **TOP_K** se crea una lista temporal con N elementos y se realizan llamadas recursivas de `merge sort`, lo cual introduce un factor adicional de memoria $O(N)$ en el peor caso, debido a las listas intermedias generadas durante el proceso de mezcla. Sin embargo, esta memoria adicional se libera una vez finaliza la operación, y la estructura principal de la matriz dispersa sigue siendo lineal en N .

9. Estrategia de dividir y vencer implementada

La estrategia de dividir y vencer se implementa en el módulo `merge_sort.py`, donde se desarrolla un algoritmo de **merge sort** para ordenar las posiciones no nulas por su valor de forma descendente. Esta estrategia se integra directamente en la operación TOP_K de la clase `MatrizDispersa`, por lo que no se trata de un ejemplo aislado, sino de una parte esencial del comportamiento del sistema.

El problema que resuelve `merge sort` es ordenar una lista de tuplas $(fila, columna, valor)$ de manera que los elementos queden organizados de mayor a menor según el campo `valor`. El caso base corresponde a listas de tamaño cero o uno, que ya se consideran ordenadas, mientras que el caso recursivo divide la lista original en dos mitades, ordena cada mitad de forma recursiva y luego combina ambas listas ordenadas en una lista final también ordenada.

La complejidad temporal del algoritmo es $O(N \log N)$, producto de dividir la lista en dos mitades en cada nivel de recursión y de realizar un proceso de mezcla lineal $O(N)$ por nivel. Esta complejidad es asintóticamente mejor que la de un algoritmo ingenuo de ordenamiento cuadrático

$O(N^2)$, por lo que resulta adecuada para conjuntos de datos grandes y para el contexto del proyecto donde se valora la eficiencia de operaciones de ranking.

10. Comparación con una solución ingenua

Una solución ingenua para este problema consistiría en construir una matriz completa de tamaño $F \times C$ y almacenar en memoria todas sus posiciones, incluso aquellas cuyo valor es cero. Este enfoque resulta inviable cuando las dimensiones de la matriz son muy grandes, por ejemplo del orden de $10^9 \times 10^9$, ya que el consumo de memoria crecería proporcionalmente al producto de ambas dimensiones.

En contraste, la solución implementada en este proyecto almacena únicamente las posiciones no nulas, utilizando una lista enlazada simple donde cada nodo representa una celda relevante de la matriz. De esta forma, el uso de memoria deja de depender del tamaño total del dominio y pasa a depender exclusivamente de la cantidad de elementos realmente almacenados.

Desde el punto de vista temporal, una matriz densa permitiría consultas puntuales como GET en tiempo constante. Sin embargo, ese beneficio no compensa el altísimo costo de memoria en escenarios donde la mayoría de posiciones están vacías. En el contexto del problema, resulta más adecuado aceptar costos lineales en algunas operaciones a cambio de una representación que sí sea viable para datos dispersos.

Otra alternativa ingenua para la operación TOP_K sería ordenar los elementos usando un algoritmo cuadrático, como el ordenamiento por selección o por burbuja. Esa solución tendría complejidad $O(N^2)$, mientras que la estrategia implementada con *merge sort* reduce el costo a $O(N \log N)$, lo cual representa una mejora significativa cuando la cantidad de posiciones no nulas es grande.

Por lo tanto, la solución propuesta no solo cumple con las restricciones del enunciado, sino que además ofrece un mejor balance entre consumo de memoria, claridad de implementación y eficiencia algorítmica para el tipo de datos considerado en el proyecto.

11. Casos de prueba utilizados

Para verificar el funcionamiento del programa, se utilizaron distintos casos de prueba con tamaños y comportamientos variados. En primer lugar, se probó el ejemplo base del enunciado, con el objetivo de comprobar que el formato de lectura, el procesamiento de operaciones y la generación de la salida coincidieran con lo esperado.

Un caso de prueba utilizado fue el siguiente:

```
1000000000 1000000000 6
1 1 5
1 100 7
500 300 9
```

```

1000 1000 2
200000 10 11
999999999 999999999 15
7
GET 500 300
ROW_SUM 1
COL_SUM 100
SET 1 100 20
GET 1 100
REGION_SUM 1 1 1000 1000
TOP_K 3

```

La salida esperada para este caso es:

```

GET 500 300 = 9
ROW_SUM 1 = 12
COL_SUM 100 = 7
SET 1 100 = OK
GET 1 100 = 20
REGION_SUM 1 1 1000 1000 = 36
TOP_K 3 = (1,100,20) (999999999,999999999,15) (200000,10,11)

```

Además del caso base, también se realizaron pruebas específicas para verificar operaciones individuales. Por ejemplo, se probaron actualizaciones sobre posiciones existentes, inserciones en posiciones vacías, eliminación de nodos, consultas sobre filas y columnas sin elementos, cálculo de densidad y verificación del comportamiento de TOP_K con distintos valores de k .

También se validó el funcionamiento del programa cuando el archivo `salida.txt` ya existía previamente. En este caso, se comprobó que el programa sobrescribe correctamente el contenido del archivo al ejecutarse nuevamente, manteniendo así el comportamiento esperado de generación automática de resultados.

12. Casos límite considerados

Durante el diseño y la prueba del programa se tuvieron en cuenta varios casos límite sugeridos por el enunciado para matrices dispersas.

- **Matriz sin valores no nulos:** se consideró el caso en el que $N = 0$, verificando que operaciones como GET, ROW_SUM, COL_SUM, REGION_SUM y DENSITY devuelvan resultados coherentes.
- **Actualizar una posición existente:** se probó que SET reemplace correctamente el valor de una celda ya almacenada, sin duplicar nodos en la lista enlazada.
- **Actualizar una posición vacía:** se verificó que SET inserte un nuevo nodo cuando la posición no existe previamente.

- **Eliminar una posición inexistente:** se comprobó que la operación DELETE no cause errores y mantenga la consistencia de la estructura.
- **Consultar una celda vacía:** se validó que GET retorne 0 cuando la posición solicitada no está almacenada.
- **Regiones sin valores:** se probó que REGION_SUM retorne 0 cuando ningún nodo pertenece al rectángulo consultado.
- **Regiones que cubren toda la matriz:** se verificó que REGION_SUM pueda sumar correctamente todos los valores no nulos cuando el rango incluye la totalidad del dominio.
- **Valores negativos:** se tuvo en cuenta que la matriz puede almacenar enteros negativos y que estos deben afectar correctamente las sumas y el ordenamiento en TOP_K.
- **Coordenadas grandes:** se utilizaron coordenadas muy altas para confirmar que la implementación no depende del tamaño físico de la matriz, sino únicamente de la cantidad de elementos no nulos almacenados.
- **Valor k mayor al número de elementos:** se verificó que TOP_K devuelva todos los elementos disponibles cuando k supera la cantidad real de posiciones almacenadas.

La consideración de estos casos permitió depurar la lógica del programa y asegurar que el comportamiento de la solución fuera consistente no solo en ejemplos simples, sino también en escenarios extremos o poco frecuentes.

13. Conclusiones

El desarrollo de este proyecto permitió implementar una solución funcional para el procesamiento de matrices dispersas, respetando las restricciones del enunciado y aplicando varios de los conceptos fundamentales vistos en el curso.

La representación basada en lista enlazada simple resultó adecuada para modelar una matriz con pocas posiciones no nulas, ya que evita completamente la construcción de una matriz densa y reduce el uso de memoria a una cantidad proporcional al número de elementos realmente almacenados. Aunque esta decisión implica costos lineales en varias operaciones, se mantiene coherente con el propósito del problema y con las restricciones sobre estructuras de datos permitidas.

La implementación de *merge sort* para resolver la operación TOP_K permitió integrar correctamente una estrategia de dividir y vencer dentro de una funcionalidad importante del sistema. Esto no solo cumple uno de los requerimientos formales del proyecto, sino que también mejora la eficiencia frente a enfoques ingenuos de ordenamiento.

Finalmente, el proyecto permitió fortalecer habilidades relacionadas con lectura y escritura de archivos planos, diseño modular, implementación propia de estructuras de datos, análisis de complejidad y justificación técnica de decisiones algorítmicas.